

Key insights

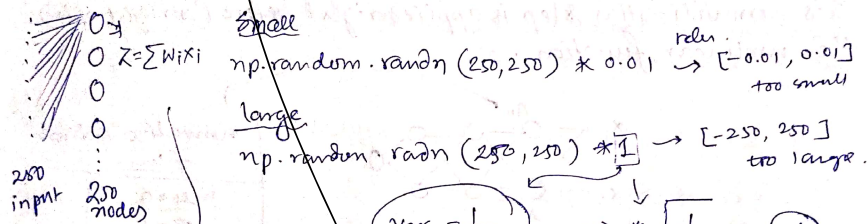
Things not to do -

- ① zero initialization. $\rightarrow$ no training
- ② Non-zero constant initialization. $\rightarrow$ linear
- ③ Random initialization with small weights. $\rightarrow$ slow convergence
- ④ Random initialization with large weights. $\rightarrow$ irregular convergence

What can be done?

$\rightarrow$ Heuristics (Jugad) $\rightarrow$ Practical Solution

Intuition



the goal is that,
If the no. of inputs are large, then make sure that the z will not be too large.
and if no. of inputs less, then make sure the value of z will not be less.

$$\text{Var} = \frac{1}{n} \rightarrow \cdot \sqrt{\frac{1}{250}} \rightarrow \text{small}$$

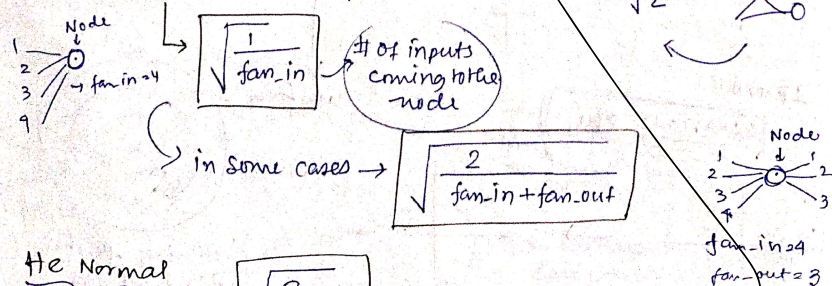
inputs of the previous layer

Suppose $n_{ip} = 250 \rightarrow \cdot \sqrt{250} \rightarrow \text{large}$

$n_{ip} = 5 \rightarrow \cdot \frac{1}{\sqrt{5}} \rightarrow \text{small}$

Xavier Initialization. (applicable in tanh)

(normal) $\rightarrow$ $\text{np.random.randn}(2, 2) * \frac{1}{\sqrt{2}}$



He Normal (normal) $\rightarrow \sqrt{\frac{2}{\text{fan.in}}}$

Xavier (uniform)

Uniform distribution

He (uniform)

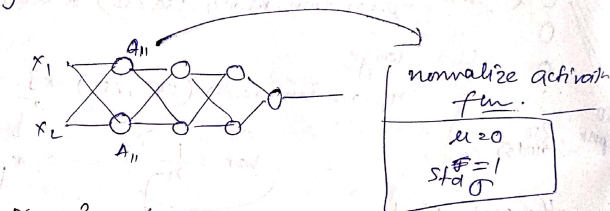
$$\lim = \sqrt{\frac{6}{(\text{fan.in} + \text{fan.out})}}$$

$$\lim = \sqrt{\frac{6}{\text{fan.in}}}$$

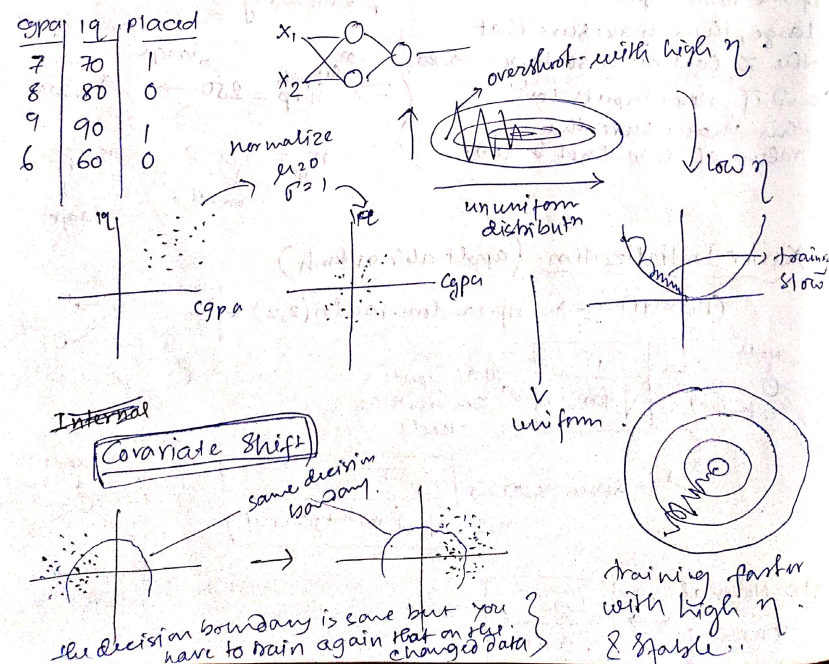
What is Batch Norm?

Batch Normalization (BN) is an algorithmic method which makes the training of DNN faster & more stable.

It consists of normalizing activation vectors from hidden layers using the mean and variance of the current batch. This normalization step is applied right before (or right after) the nonlinear function.

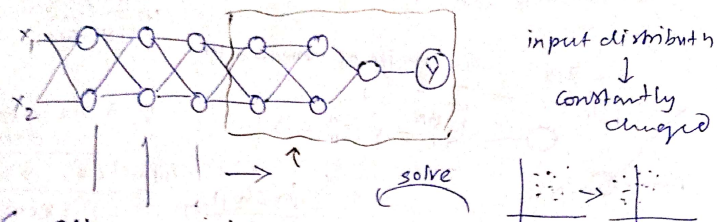


Why use Batch Norm?



Internal Covariate Shift.

We defined Internal Covariate Shift as the change in the distribution of Network activations due to the change in network parameters during training.



after every layer there we will apply normalization $\mu=0, \sigma=1$, so that the distribution

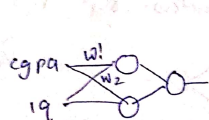
training slow & unstable.

If do not use batch normalization then you have to take η low.

26/07/25

Batch Norm - The how.

gpa	iq	placed
8.9	100	1
6.2	84	0
9.1	91	0
7.2	74	1
...	...	...
6.7	91	0



$$Z_{11} = w_1 \cdot \text{gpa} + w_2 \cdot \text{iq} + b$$

$$g(Z_{11}) = a_{11}$$

$$\text{way 1} \rightarrow Z_{11} \rightarrow Z_{11}^N \rightarrow g(Z_{11}^N) \Rightarrow a_{11}$$

↑
Normalized.

$$\frac{Z_{11} - \mu}{\sigma}$$

$$\text{way 2} \rightarrow Z_{11} \rightarrow g(Z_{11}) \Rightarrow a_{11} \rightarrow a_{11}^N$$

↑
Normalized

batch_size = 4

$$\begin{bmatrix} 8.9 & 100 \\ 6.2 & 84 \\ 9.1 & 91 \\ 7.2 & 74 \end{bmatrix} \begin{bmatrix} w_{11} & w_{12} \\ w_{21} & w_{22} \end{bmatrix} + b$$

Normalized Z_{11} 4x2

mean of 4 activations

$$\mu_B = \frac{1}{m} \sum_{i=1}^m Z_{11}^i$$

(m=4)

$$\sigma_B = \sqrt{\frac{1}{m} \sum_{i=1}^m (Z_{11}^i - \mu_B)^2}$$

$$= \begin{bmatrix} \vdots \\ \vdots \\ \vdots \\ \vdots \end{bmatrix} + b$$

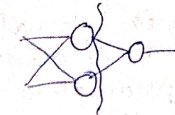
4x2 (normalized op)

↑ activations for 4 datapoints.

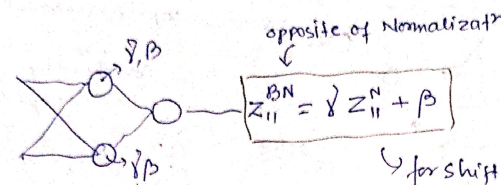
for every neuron → for single activation -

$$Z_{11}^i = \frac{Z_{11}^i - \mu_B}{\sigma_B + \epsilon}$$

error term



scale and shift

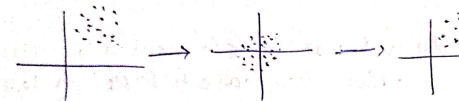


$$Z_{11} \rightarrow Z_{11}^N \rightarrow Z_{11}^{BN} \rightarrow g(Z_{11}^{BN}) \Rightarrow a_{11}$$

for shifting

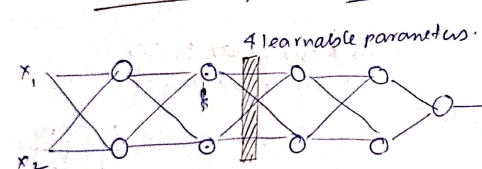
$\gamma, \beta \rightarrow$ learnable parameters
initial $\gamma=1, \beta=0$
(Keras)

flexibility
(using these 2 parameters you can have other distribution also)



Multiple layer (BN).

Neuron 1 Neuron 2



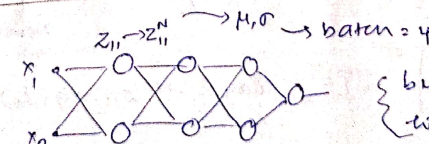
update γ

$$\gamma = \gamma - \eta \frac{\partial L}{\partial \gamma}$$

update this while backpropagation
Similarly like w, b update

Batch Normalization during test

gpa	iq	placed
8	90	1
7	70	0
6	60	1
...	...	...
9	90	1



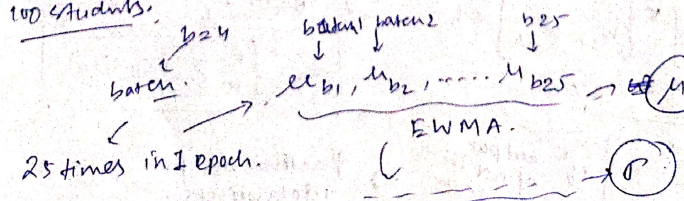
but in testing we have 1 query pt

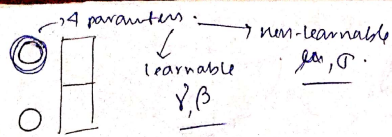
how to get μ, σ ?

Exp weighted moving avg.

EWMA

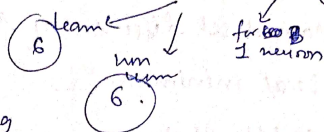
100 students





while training

total parameters = $3 \times 4 = 12$



Advantages

1) Stable → hyperparameter tuning
→ wider range of values.

2) faster training → $\eta \uparrow$

3) Regularizer → μ, σ randomness noise overfitting ↓

dropout reg.

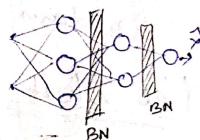
side +ve impact of Batch normalization.

4) wt. init impact reduced.



Keras Implementation

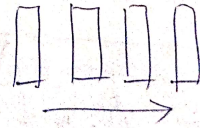
```
model.add(Dense(3, activation='relu', input_dim=2))
model.add(BatchNormalization())
model.add(Dense(2, activation='relu'))
model.add(BatchNormalization())
model.add(Dense(1, activation='sigmoid'))
```



Optimizers in Deep Learning → Speed up in NN training.

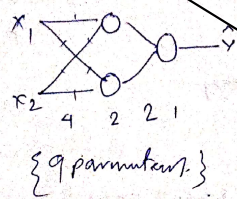
→ Weight init ✓
→ Batch Norm ✓
→ Activation fn ✓

Optimizers ✓



Role of Optimizer

type	id	placed
8	80	1
9	90	1
7	70	0

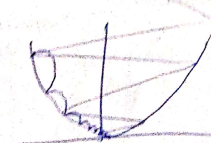


optimization
2 min.
 $\gamma, \hat{\gamma} (w, b)$

{9 parameters}

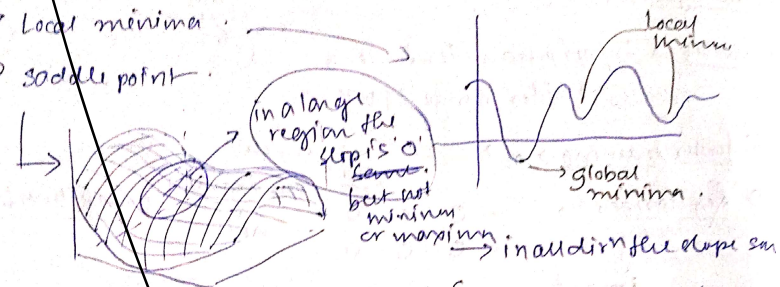
Challenges in GD

- 1) Learning Rate $w_1 = w_0 - \eta \frac{\partial L}{\partial w_0}$
- 2) Learning Rate Scheduling → pre define
→ before training.
- 3) cannot set different η for different w .



4) Local minima.

5) Saddle point.

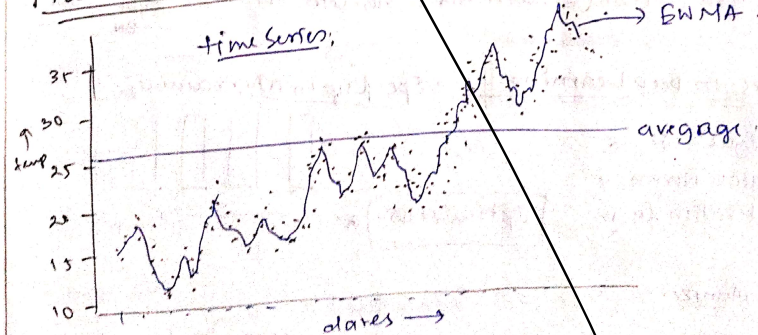


for flux cause, we have to know new optimizers

What next?

- 1) Momentum
 - 2) Adagrad
 - 3) NAG
 - 4) RMS prop
 - 5) Adam
- Exponentially Weighted Moving Avg.

The What (EWMA)



usages

- time series forecasting
- financial
- signal processing
- Deep learning - optimizers.

EWMA

D1	0	weightage
D2	0	
D3	0	
D4	0	weightage